Frame

# Algorithmen und Datenstrukturen

Sommersemester 2026

Jana Grajetzki (jana.grajetzki@uni-jena.de)

## Aufgabenblatt 3

Abgabe Donnerstag, den 7.5.2026, bis 12.00 Uhr. Bitte laden Sie Ihre Lösungen in Moodle als pdf-file hoch.

Begründen Sie Ihre Antworten immer, auch wenn dies nicht explizit gefordert wird.

**Aufgabe 1:**
Gegeben ist ein ungeordnetes Array `A[1,...,n]` mit ganzen Zahlen.
Geben Sie einen Algorithmus an, der in einem Zug das Maximum und das Minimum bestimmt und möglichst wenige Vergleiche ausführt.

(6 Punkte)

**Aufgabe 2:**

(a) Wenden Sie Build-Max-Heap(A) auf das Array
$A = (6, 2, 7, 11, 16, 14, 4, 8, 9, 12, 22, 1, 3, 5)$ an.
(Bitte Baumdarstellung zur Veranschaulichung.)

(b) Führen Sie die Operation Heap-Extract-Max(A) drei mal aus. (8 Punkte)

**Aufgabe 3:**
Beantworten Sie folgende Fragen und begründen Sie Ihre Antwort:

(a) Stellt das Array $A = [33, 26, 14, 26, 5, 15, -3, 26, 9, 3]$ einen Heap dar?

(b) Ist jedes absteigend sortierte Feld ein Heap?

(c) Wir bertrachten Max-Heaps in denen alle Schlüssel paarweise verschieden sind. Wie kann in einem solchen Heap das Minimum gefunden werden? Welche Zeit benötigt Ihr Algorithmus? Geben Sie eine untere Schranke für das Finden des Minimums in einem binären Heap an.

(8 Punkte)

**Aufgabe 4:**
Betrachten Sie folgende Funktion zum Aufbau eines Heaps:

```
BH'(A)
  heap_size[A] = 1
  for i=2 to length(A) do
    HEAP_INSERT (A, A[i])
```

Beantworten Sie folgende Fragen und begründen Sie Ihre Antwort:

(a) Erzeugen BUILD-MAX-HEAP(A) und BH'(A) für jedes Feld A den gleichen Heap?

(b) Zeigen Sie, dass die asymptotische Laufzeit von BH'(A) im schlechtesten Fall $\Omega(n \log n)$ beträgt.

(8 Punkte)

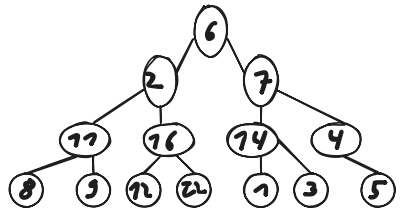
Aufg. 1

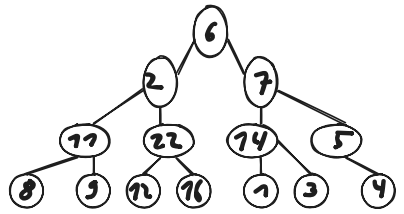
1. Wenn n gerade:
   vergleiche A[1] und A[2]
   min = kleineres, max = größeres
   i = 3
   Sonst: min = max = A[1]
   i = 2
2. Für i bis n in Schritten von 2:
   vergleiche A[i] und A[i+1]
   kleines → mit min vergleichen
   größeres → mit max vergleichen
   Pro Paar: 3 Vergleiche
   $\approx \frac{3n}{2}$

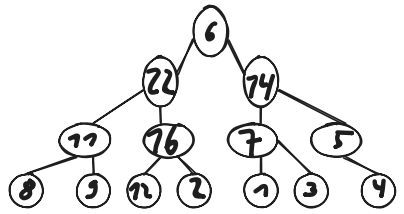
Aufg. 2

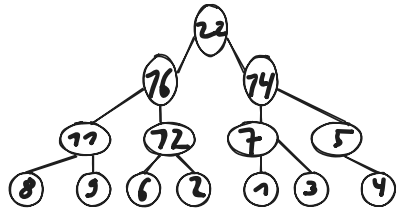
a)

A = [6, 2, 7, 11, 16, 14, 4, 8, 9, 12, 22, 1, 3, 5]



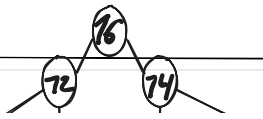



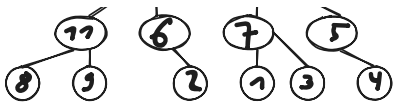




b)

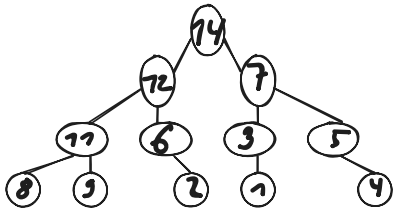
1.

[16, 12, 14, 11, 6, 7, 5, 8, 9, 2, 4, 1]

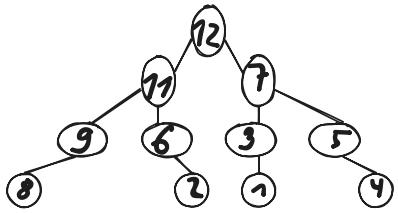





2.




3.




Nr 3

a) Ist kein Heap da 14 als Kind 15 hat

b) Ja Jedes absteigende Sortierte Feld ist ein Heap da Automatisch der Elternknoten immer größer als das Folgeglied ist und somit auch größer als der Kinderknoten

c) Minimum ist in den Blättern

$\lfloor \frac{n}{2} \rfloor + 1$ bis $n$

Laufzeit: $O(n)$

naiver Schranken: $\Omega(n)$
da man alle Blätter anschauen muss

Nr. 4

a) Die Struktur kann unterschiedlich sein
da:
Build-Max-Heap: Bottom-Up
BH' : Ein fügen und sortieren

b) Sei $A$ ein streng Aufsteigend sortiertes Feld $A = (1, 2, \dots, n)$

-> Dann ist Jedes neu hinzugefügte Element größer als alle bisherigen und steigt beim Heap-INSERT Jeweils bis zur Wurzel, Das kostet für den $i$-th Element im Worst Case $\Theta(\log i)$ Zeit.

Gesamtlaufzeit wird: $\sum_{i=2}^{n} \Omega(\log i)$

$\geq \Omega(n \log n)$